

From Curation to Code: A Practical Framework for Building Single-Domain Datasets for Low-CPU LLM Training

Strategic Foundations: Domain Selection and Curated Sourcing

The development of a successful Large Language Model (LLM) is fundamentally tied to the quality and specificity of the data upon which it is trained [76](#). This report outlines a comprehensive framework for creating high-quality, single-domain datasets tailored for efficient training on low-CPU systems. The core strategy revolves around two foundational pillars: a disciplined approach to domain specialization and a rigorous commitment to curated sourcing over large-scale, unfiltered scraping. The user's explicit mandate to focus on one domain at a time—be it general knowledge, technical documentation, conversational dialogue, or code generation—is not merely a structural choice but a strategic decision with significant implications for model performance, computational cost, and overall efficiency [143](#). By concentrating on a single domain, developers can cultivate deep expertise in a specific area, leading to models that outperform their larger, more generalized counterparts on targeted tasks [26](#) [28](#). Fine-tuning with domain-specific data has demonstrated performance improvements ranging from 18.3% to 42.7% across various benchmarks, underscoring the value of specialization [26](#). This approach directly counters the trade-off between generalization and specialization; while combining diverse datasets can create a broadly capable model, it may lack the nuanced understanding of a model trained on a highly focused corpus [143](#). For practitioners working within the constraints of low-CPU environments, this specialization becomes even more critical, as it allows for the creation of smaller, more efficient models that do not require the immense computational resources needed to train massive, multi-domain systems from scratch [17](#) [67](#).

The principle of prioritizing data quality over sheer quantity is paramount throughout this framework [76](#) [144](#). Research indicates that data quality is as important as data quantity, and in some cases, a smaller, meticulously curated dataset can yield better results than a much larger, noisier one [144](#). The construction of high-quality datasets often

begins with identifying authoritative and reliable sources. For general knowledge, this involves leveraging structured, factual information from sources like legal and financial databases, scientific literature archives, and established encyclopedias [56](#). For instance, the Common Corpus project builds its foundation on legally compliant sources such as government financial reports from the SEC, legal texts from Caselaw Access Project, and scholarly articles from OpenAlex, all under permissive licenses [56](#). Similarly, for technical documentation, the primary sources are official API references, product manuals, and well-documented open-source codebases [76](#) [96](#). In the realm of conversational dialogue, high-quality data is typically derived from human-annotated corpora where conversations were generated based on strict guidelines to ensure coherence and naturalness, rather than from raw, unfiltered web crawls [36](#) [115](#). For code generation, the gold standard is datasets containing verified, function-level code-documentation pairs or solutions to programming problems that have been automatically tested for correctness [41](#) [96](#). These examples illustrate a consistent pattern: the most valuable datasets are those built from the ground up using trusted, well-defined sources that align with the target domain's requirements for accuracy, structure, and relevance.

The concept of "curated sourcing" extends beyond simply choosing reputable websites or repositories. It encompasses a systematic process of collection, validation, and refinement designed to maximize signal and minimize noise. This process must account for several key factors that impact both the utility and legality of the data. One of the most significant concerns is data contamination, which occurs when training data inadvertently includes content from downstream evaluation benchmarks or proprietary sources [80](#). Such contamination can lead to overly optimistic and misleading benchmark scores, as the model has effectively memorized the answers rather than learning to solve the problems anew [79](#). Therefore, a robust curation framework must include a decontamination stage to proactively identify and remove any shared samples between the training set and public test sets [77](#). Another critical aspect of curation is ensuring privacy compliance by detecting and redacting Personally Identifiable Information (PII) from the text [57](#). Tools like Microsoft Presidio can be integrated into the pipeline to automatically scan for and replace PII, mitigating ethical and legal risks associated with data leaks [56](#). Furthermore, adherence to industry standards, such as ISO/IEC standards for software engineering processes, can lend credibility and consistency to the curated data, particularly in technical domains [125](#) [126](#). By embedding these principles of quality, integrity, and compliance into the very beginning of the data lifecycle, the framework establishes a solid foundation for building specialized models that are not only effective but also trustworthy and safe. The emphasis on curated sources ensures that the resulting model learns from reliable, relevant, and ethically sourced information, a crucial factor

for success in any application, especially when computational resources are limited and every data point must contribute meaningfully to the final outcome.

The Multi-Stage Data Curation Pipeline

A robust data curation pipeline is the central mechanism for transforming raw, potentially noisy source materials into a high-quality, single-domain dataset suitable for efficient LLM training. This pipeline is not a monolithic block but a sequence of logically distinct stages, each designed to progressively refine the data, enforce domain discipline, and optimize it for the subsequent training phase. The entire process must be adaptable to the nuances of each target domain—from general knowledge to code generation—but adheres to a common set of core principles aimed at maximizing data utility and minimizing computational overhead. The first stage, **Acquisition and Initial Filtering**, involves systematically downloading data from the pre-identified high-quality sources and performing preliminary cleaning. This initial pass is crucial for removing bulk, irrelevant content and reducing the scale of the data before more intensive processing begins. Frameworks like NeMo Curator provide tools for downloading data from common sources such as Common Crawl snapshots or Wikipedia dumps, and they offer helpers for basic text extraction and preparation [57](#). During this stage, files are filtered based on criteria like file extensions, size, and licensing agreements to exclude non-textual data, excessively large files that could cause processing bottlenecks, and content with restrictive usage rights [56](#) [77](#). For example, when curating an open-code dataset, non-informative file types like CSV or JSON might be explicitly excluded from the start [56](#).

Following acquisition, the second stage, **Domain Isolation and Classification**, acts as the gatekeeper for the single-domain mandate. This step employs either heuristic-based rules or machine learning classifiers to sort through the acquired data and retain only what falls within the predefined scope of the target domain [57](#). This is a critical control point to prevent cross-domain contamination, which can confuse the model and dilute its specialized capabilities. For instance, a dataset intended for technical documentation should be rigorously screened to remove any snippets of conversational dialogue or unrelated general knowledge. NeMo Curator facilitates this by allowing users to define custom filters for domain classification, enabling a flexible yet powerful way to enforce scope [57](#). The third stage, **Quality Scoring and Filtering**, introduces a more granular layer of quality assurance. Here, the remaining data is evaluated against a rubric specifically designed for the domain. This moves beyond simple binary classification to a more nuanced assessment of value. A prime example is the Code2Doc dataset, which uses

a sophisticated scoring system based on a weighted average of eight dimensions, including completeness, clarity, parameter coverage, and use of type annotations [69](#). Only samples scoring above a certain threshold (e.g., $\geq 6.0/10$) are retained for the final dataset [69](#). For conversational data, quality metrics might include measures of coherence, topic consistency, and naturalness, as enforced in the NaturalConv dataset where dialogues were checked for repetition and manually sampled for approval [36](#). This domain-specific quality filtering ensures that the final dataset is composed of high-value examples that will teach the model effectively.

The fourth and fifth stages, **Deduplication and Decontamination**, address fundamental issues of data redundancy and integrity. Deduplication is a non-negotiable step in any serious curation effort. Duplicated data inflates the apparent size of the training set without providing new information, which can lead to model overfitting and memorization of specific patterns [65](#) [155](#). More importantly, it wastes valuable computational resources during training. The literature distinguishes between local deduplication (removing duplicates within a single source) and global deduplication (removing duplicates across all combined sources) [143](#). Global deduplication is generally preferable because it eliminates redundant information that might exist across different source datasets, but it requires more memory [143](#). To make this computationally expensive task manageable, GPU-accelerated methods are essential. Frameworks like NeMo Curator utilize optimized CUDA implementations of algorithms such as MinHash and Locality-Sensitive Hashing (LSH) to efficiently identify and remove near-identical documents [57](#) [133](#). Decontamination follows a similar logic but targets a different problem: preventing data leakage from downstream evaluation sets. This involves checking the candidate training data against public benchmarks and removing any exact matches to ensure that model performance reflects true learning capability rather than memorization of test items [77](#). The final stage of the pipeline is **Privacy Compliance**, which focuses on ethical and legal considerations. This involves using automated tools like Microsoft Presidio to detect and redact Personally Identifiable Information (PII), such as names, addresses, and other sensitive details, from the text corpus [56](#). This proactive measure protects user privacy and helps mitigate potential legal liabilities, making the dataset safer and more responsible to use. Together, these five stages form a comprehensive and systematic workflow for building a clean, focused, and high-quality dataset that is primed for efficient and effective model training.

Curation Stage	Primary Goal	Key Techniques & Tools	Domain-Specific Application
Acquisition & Initial Filtering	Reduce data volume and remove irrelevant content early.	Source identification, file extension/size filtering, license checks 56 , frameworks like NeMo Curator 57 .	For code, filter out non-source files (e.g., .csv, .json) 56 ; for knowledge, filter out non-textual media.
Domain Isolation & Classification	Enforce the single-domain mandate by removing cross-domain content.	Heuristic rules, ML classifiers, custom domain filters 57 .	Ensure a conversational dataset contains no code snippets; a technical doc set contains no chit-chat.
Quality Scoring & Filtering	Retain only the highest-value examples based on domain-specific criteria.	Weighted scoring rubrics, expert review, semantic similarity measures 8 , AI-generated content detection 69 .	For code, score for clarity and correctness; for conversation, score for coherence and naturalness 36 .
Deduplication	Eliminate redundant data to improve training efficiency and prevent overfitting.	Global vs. local deduplication 143 , MinHash, Locality-Sensitive Hashing (LSH), GPU acceleration 133 .	Use MinHash to find near-duplicate code functions or identical encyclopedia entries across multiple sources.
Decontamination	Prevent data leakage from downstream evaluation benchmarks.	Comparison against public test sets, removal of exact matches 77 .	Remove any sample from the training set that appears in a benchmark like HumanEval or MultiWOZ2.1.
Privacy Compliance	Protect user privacy by removing Personally Identifiable Information (PII).	Automated PII detection and redaction tools (e.g., Microsoft Presidio) 56 .	Scan all text for names, email addresses, and other sensitive data before inclusion in the dataset.

Preprocessing and Formatting for Low-CPU Efficiency

Once the curated dataset has been refined through the multi-stage pipeline, the final preparatory phase focuses on formatting the data for maximum efficiency during training, particularly on low-CPU systems. The computational bottleneck in such environments is often not raw processing speed but memory bandwidth—the rate at which data can be read from and written to memory [53](#). Therefore, every step in this final stage must be designed to minimize data size, reduce I/O operations, and streamline the data loading process. The most critical aspect of this phase is the selection of an appropriate data format. The widely adopted standard for LLM fine-tuning is JSON Lines (JSONL) [76](#). In this format, each line in a text file represents a single, self-contained training example, typically a JSON object. This structure offers several advantages for resource-constrained training. First, it enables efficient streaming of data directly from disk during the training loop, eliminating the need to load the entire dataset into RAM at once [76](#). This is crucial for low-RAM systems. Second, its simple, line-oriented nature makes it easy to process, split, and manage at scale. Major deep learning libraries like Hugging Face Transformers and PEFT have excellent native support for the JSONL

format, making it the de facto industry standard [76](#). The massive SlimPajama dataset, for instance, is distributed as approximately 59,000 individual JSONL files, highlighting the modularity and scalability of this approach [142154](#). Using a standardized format like JSONL simplifies the integration with existing training pipelines and reduces the overhead of custom data loaders.

Beyond the container format, the internal structure of the data itself is paramount. The data must be transformed into a schema that is conducive to Supervised Fine-Tuning (SFT), the primary method for adapting a pre-trained LLM to a specific domain [48](#). There are three principal styles for structuring fine-tuning data, and the choice depends on the target task [76](#). The **Completion-Style** format pairs a prompt with a desired completion (e.g., `prompt + answer`). This style is best suited for narrow, specific tasks where the context is minimal [76](#). The **Instruction-Style** format is more versatile and is recommended for most domain adaptation scenarios. It typically uses a standardized template that separates the instruction, an optional input or context, and the expected response. A common structure is:

```
### Instruction:  
<Task description>
```

```
### Input:  
<Optional domain context>
```

```
### Response:  
<Expected output>
```

This clear separation helps the model understand its role and the required output format, and it is compatible with many popular fine-tuning datasets like Alpaca and Dolly [76](#). Finally, the **Chat-Style** format is essential for training conversational agents. It structures the data as a sequence of role-based messages (e.g., `system`, `user`, `assistant`), mimicking a real-world dialogue [76](#). For a developer following this framework, selecting the correct style is the first step in shaping the model's behavior. For example, a dataset for technical documentation would benefit from the Instruction-Style format, where the instruction is "Explain the functionality of this API endpoint," the input is the code snippet, and the response is the generated documentation. Conversely, a dataset for a customer service bot would require the Chat-Style format to learn turn-taking and context management.

For extremely resource-constrained environments where even the curated and formatted dataset might be too large to process efficiently, advanced techniques for data

optimization become necessary. One such approach is **Dataset Condensation (DC)**, which aims to create a small, synthetic dataset that retains the essential information of the original, much larger set [75](#). Models trained on this condensed dataset can achieve performance comparable to those trained on the full data, drastically reducing the amount of data that needs to be processed during training [75](#). Another powerful technique is influence-based data selection, exemplified by methods like BLISS (Bilevel Influence Scoring for Data Selection) [23](#) [24](#). These algorithms operate entirely from scratch to identify a small subset of the most informative training samples, allowing a model to learn effectively from a fraction of the data [24](#). Other lightweight selection strategies include entropy-based filtering, which identifies and removes less informative or redundant samples based on their statistical properties [54](#). These methods represent a paradigm shift from accumulating vast amounts of data to strategically selecting a minimal, high-impact set. When combined with parameter-efficient fine-tuning methods, these lightweight datasets can enable the training of specialized models on hardware with very limited CPU and memory resources [77](#). By thoughtfully considering the data format, structuring it for the target task, and optionally employing advanced selection techniques, this final phase ensures that the dataset is not only high-quality but also optimally prepared for the unique challenges of low-CPU training.

Domain-Specific Implementation Blueprint

While the overarching framework of curated sourcing and multi-stage curation provides a universal blueprint, its implementation must be tailored to the unique characteristics and quality signals of each specific domain. A one-size-fits-all approach is insufficient for achieving the depth of specialization required for high-performance models. Each domain—general knowledge, technical documentation, conversational dialogue, and code generation—demands a distinct set of source materials, quality metrics, and curation strategies. For the **General Knowledge** domain, the primary objective is to imbue the model with a broad base of factual, structured, and verifiable information. The ideal sources are authoritative and comprehensive knowledge repositories. Foundational datasets like Wikipedia dumps serve as a starting point, but they require careful processing to extract clean text and remove navigational elements [56](#). To build a truly high-quality dataset, one should look towards specialized collections. The 'Open Government' and 'Open Science' collections within the Common Corpus project are exemplary, providing access to over 407 billion tokens of structured data from financial reports, legal documents, and scientific papers, all under permissive licenses [56](#). The

curation process here focuses on factual accuracy, temporal relevance (if applicable), and logical coherence. Quality can be assessed by cross-referencing facts against trusted external sources, a principle echoed in efforts to align LLMs to quote from pre-existing knowledge bases to enhance verifiability [109](#). The goal is to create a dataset that teaches the model not just trivia, but a structured understanding of the world, akin to the knowledge tested in professional qualification examinations [149](#).

For the **Technical Documentation** domain, the focus shifts from general facts to precise, actionable information about software, APIs, and systems. High-quality sources are official documentation, API reference guides, and well-maintained open-source codebases with detailed comments and README files [76](#). The curation process must prioritize clarity, accuracy, and up-to-date status. A notable example is the Code2Doc dataset, which curates high-quality, function-level code documentation pairs from languages like Python, Java, and C++ [69](#) [96](#). Its methodology provides a template for this domain: start with a large pool of candidates, apply a multi-stage filtering process, and use a detailed scoring rubric. The rubric for Code2Doc assesses dimensions like completeness, parameter coverage, and clarity, retaining only samples that meet a high bar for quality [69](#). This approach contrasts with simply scraping entire repositories; instead, it isolates the most valuable units of information—well-written function descriptions—and structures them for direct training. For domain-specific technical areas, such as low-resource programming languages (LRPLs) or domain-specific languages (DSLs), the curation process becomes more challenging due to data scarcity. In these cases, sources extend beyond public repositories to include textbooks, man pages, specialized forums, and even internal corporate documentation, if available [77](#). The key is to gather a dense concentration of high-quality examples within the niche domain to allow the model to learn its unique syntax and semantics effectively.

The **Conversational Dialogue** domain presents a different set of challenges, where quality is defined by attributes like coherence, relevance, empathy, and naturalness. The highest-quality datasets are typically created through human annotation guided by strict protocols. The NaturalConv dataset, for example, recruited annotators to generate long-form conversations based on a shared news article, enforcing rules that promoted natural topic shifts and allowed for chitchat, resulting in more human-like interactions [36](#). Human evaluation consistently shows a much stronger correlation with perceived dialogue quality than automated metrics like BLEU or ROUGE, which often fail to capture nuance [115](#). Therefore, the curation process should prioritize datasets that have undergone human review or use methodologies designed to produce human-like qualities. Persona-based conversational datasets, such as Synthetic-Persona-Chat, are another powerful source, as they teach the model to maintain a consistent character and

engage in more believable interactions [9](#) [93](#). LLM-assisted generation can also be used to expand seed datasets, creating large volumes of high-quality, persona-consistent conversations [94](#) [112](#). For task-oriented applications, such as customer service or travel booking, the focus shifts to datasets like Banking77 or MultiWOZ2.1, which contain structured dialogues with explicit intents and slots that are essential for training effective agents [102](#). The curation strategy must align with the intended conversational style: chit-chat requires open-domain, engaging data, while task-oriented dialogue needs structured, goal-directed exchanges.

Finally, the **Code Generation** domain places the highest premium on functional correctness and adherence to best practices. The most effective datasets are those that guarantee the code they contain works as intended. The AICoderEval benchmark exemplifies this by first generating thousands of code files and then executing them in a sandboxed environment; only files that passed at least one test case were kept, ensuring a baseline of correctness [41](#). This empirical validation step is far superior to relying on syntactic correctness alone. Another high-quality approach is to curate function-level documentation pairs, as seen in the Code2Doc dataset, which provides a direct mapping from a code snippet to its intended purpose [69](#) [96](#). The curation process for code must also consider quality beyond mere functionality. The ISO/IEC 25010 standard provides a comprehensive framework for evaluating software quality, encompassing functional correctness alongside non-functional characteristics like performance efficiency, security, and maintainability [111](#). While academic research often focuses heavily on security and performance, industry practitioners emphasize maintainability and readability as critical for long-term software health [111](#). Therefore, a truly high-quality code dataset should ideally incorporate criteria related to code readability and structure. A novel data-cleaning pipeline demonstrated that transforming existing programs by renaming variables, modularizing complex code, and adding natural language annotations significantly improved code generation performance, suggesting that cleaned and structured data is inherently more valuable for training [58](#). The curation strategy for code must therefore move beyond simple token counts and embrace a holistic view of quality that includes correctness, readability, and adherence to domain-specific best practices.

Domain	Primary Goal	Recommended Sources	Key Quality Metrics	Curation Strategy Example
General Knowledge	Factual accuracy, structured understanding.	Wikipedia dumps, legal/financial databases (SEC, EUR-Lex), scientific literature (OpenAlex) 56 .	Factual accuracy, coherence, source reliability.	Build a dataset from structured sources like the 'Open Government' collection, focusing on verifiable statements 56 .
Technical Documentation	Clarity, precision, and actionability.	Official API docs, product manuals, well-documented open-source repos (GitHub) 76 96 .	Completeness, clarity, conciseness, technical accuracy.	Curate function-documentation pairs using a multi-stage filter, scoring for completeness and clarity as in the Code2Doc dataset 69 .
Conversational Dialogue	Coherence, relevance, naturalness, and engagement.	Human-annotated corpora (NaturalConv 36 , HUMOD 115), persona-based datasets (PersonaChat 102), task-oriented datasets (MultiWOZ2.1 102).	Conversational consistency, coherence, naturalness, empathy.	Generate personas for LLMs to create diverse, multi-turn conversations, ensuring consistency and avoiding contradictions 93 129 .
Code Generation	Functional correctness, readability, and adherence to best practices.	Verified solutions (AICoderEval 41), function-documentation pairs (Code2Doc 96), LRPL/DSL sources (man pages, forums) 77 .	Functional correctness (Pass@k), readability, security, maintainability 111 .	Execute code in a sandbox to validate correctness, as done in AICoderEval, and use static analysis tools to check for maintainability issues 41 111 .

Advanced Techniques for Data Optimization and Model Adaptation

To fully realize the goal of training high-quality models on low-CPU systems, the framework must extend beyond basic data preparation into the realms of advanced data optimization and efficient model adaptation. These techniques are designed to break the traditional link between model performance and massive computational budgets, enabling practitioners with constrained hardware to achieve competitive results. A cornerstone of this approach is the use of **Parameter-Efficient Fine-Tuning (PEFT)** methods. Instead of updating the billions of parameters in a large pre-trained model—a process that is computationally prohibitive on low-CPU systems—PEFT methods freeze the original model weights and inject a small number of trainable parameters [63](#). The most prominent of these is Low-Rank Adaptation (LoRA), which has proven to be exceptionally effective for domain adaptation [32](#) [77](#). Studies show that LoRA can achieve 80-90% of the performance of full fine-tuning while requiring only 10-20% of the computational cost, making it an ideal fit for resource-constrained scenarios [77](#). Other PEFT methods like adapters also drastically reduce the number of trainable parameters, offering a practical solution for tailoring large models to specific tasks without the need

for extensive retraining [27](#) [32](#). These methods are particularly well-suited for the single-domain datasets produced by this framework, as they excel at transferring specialized knowledge from a small, high-quality dataset to the base model. The combination of a focused dataset and a PEFT method creates a synergistic effect, maximizing the impact of every training example while minimizing the computational burden.

Complementing PEFT is the strategic application of **quantization**, a technique that reduces the numerical precision of the model's weights. Standard LLMs use 32-bit floating-point numbers (FP32), but quantization can convert these to lower-precision formats like 16-bit (FP16/BF16), 8-bit integers (INT8), or even 4-bit integers (INT4) [86](#). This reduction has profound effects on efficiency. Lower-precision models have a significantly smaller memory footprint, which is critical for running on devices with limited RAM [50](#). Furthermore, computations with smaller integers are faster on modern CPUs, directly increasing inference throughput [81](#). Research has shown that post-training quantization (PTQ) can be effectively applied to LLMs deployed on edge devices like the Raspberry Pi, enabling high-throughput and energy-efficient execution [81](#) [84](#). While aggressive quantization can lead to some loss in accuracy, comprehensive studies have found that models can maintain strong performance even at INT8 and INT4 precisions across a wide range of benchmarks [86](#). For a developer following this framework, the recommended workflow would be to first fine-tune the model using a PEFT method like QLoRA (which combines LoRA with quantization of the base model weights) and then potentially apply further post-training quantization to the final model. This layered approach to optimization can dramatically shrink the model's resource requirements, making it feasible to run and deploy specialized models on low-power hardware [50](#).

In addition to optimizing the model itself, the training process can be made more efficient through intelligent data selection and context management. As previously mentioned, techniques like Dataset Condensation (DC) and influence-based selection (e.g., BLISS) allow for the creation of a minimal yet highly representative subset of the full curated dataset [23](#) [75](#). This is particularly useful when training time or CPU cycles are severely limited. Instead of iterating over millions of tokens, the model can be trained on a condensed set of a few thousand or tens of thousands of examples that have been proven to be maximally informative [75](#). Another area of optimization lies in managing the model's context window. Long-context tasks can be computationally expensive, but recent innovations like Structured Packing (as seen in the SPLICE algorithm) have shown that improving the utilization of the context window can lead to significant performance gains on tasks like the "needle in a haystack" benchmark [85](#). Furthermore, research into context compression explores ways to distill the essential information from long documents, reducing inference costs while preserving downstream performance [40](#).

While these techniques are more advanced, they represent the frontier of efficiency. For the practitioner aiming to train on low-CPU systems, a practical takeaway is to focus on creating the smallest possible high-quality dataset and to leverage PEFT and quantization to its fullest extent. By combining a carefully curated single-domain dataset with these advanced optimization techniques, it becomes possible to develop highly capable, specialized language models that can operate effectively in environments where large-scale computing is not an option.

Synthesis: An Actionable Methodology for Specialized Model Training

This research culminates in a cohesive, actionable methodology for developing high-quality, single-domain datasets to enable efficient Large Language Model training on low-CPU systems. The proposed framework, termed the **Curate-and-Focus Methodology**, integrates the principles of domain specialization, curated sourcing, systematic curation, and resource-aware optimization into a single, repeatable process. It directly addresses the user's goal by providing a clear pathway from raw information sources to a finely tuned, domain-specific model, all while respecting the critical constraint of limited computational power. The methodology is not a rigid prescription but a strategic guide that empowers developers to build effective models without requiring access to expensive, multi-GPU infrastructure. Its strength lies in its disciplined, multi-phase structure, which prioritizes data quality and domain purity at every step, ensuring that the final model is not just trained, but genuinely specialized.

The methodology can be distilled into four sequential phases. **Phase 1: Define and Isolate** establishes the strategic foundation. It begins with the deliberate selection of one of the four specified domains and the creation of a detailed scope document that explicitly defines acceptable and unacceptable data types. This is immediately followed by the establishment of a domain-specific quality rubric, moving beyond generic notions of "good" data to concrete, measurable criteria such as factual accuracy for knowledge, functional correctness for code, or conversational coherence for dialogue. This phase enforces the single-domain mandate from the outset, preventing the subtle but damaging effects of data contamination that can undermine model performance and evaluation integrity [80](#) [143](#).

Phase 2: Curate and Filter operationalizes the sourcing and refinement process. This phase leverages high-quality, authoritative sources identified for the chosen domain, such

as legal databases for knowledge, official API docs for technical content, or verified solutions for code [41](#) [56](#) [76](#). The data undergoes a rigorous, multi-stage filtering process: initial cleaning to remove noise, domain classification to enforce scope, quality scoring based on the established rubric, and global deduplication to eliminate redundancies and prevent overfitting [57](#) [69](#) [143](#). This systematic purification is the most critical step in ensuring that the dataset is composed of high-signal, low-noise examples, maximizing the learning efficiency of the training process [144](#).

Phase 3: Prepare and Format readies the dataset for training. The cleaned and filtered data is structured into a format optimized for Supervised Fine-Tuning, with the Instruction-Style format being a versatile choice for most domain adaptation tasks [76](#). The final dataset is packaged in the standard JSON Lines (JSONL) format, which enables efficient data streaming and is widely supported by modern ML libraries [76](#). For situations of extreme resource limitation, this phase also includes an optional step of applying a lightweight data selection algorithm to distill the curated dataset into its most impactful subset, further reducing the computational load [54](#) [75](#).

Phase 4: Train and Validate brings the methodology to fruition. The prepared dataset is used to fine-tune the pre-trained LLM using Parameter-Efficient Fine-Tuning (PEFT) methods, specifically Low-Rank Adaptation (LoRA) or its quantized variant, QLoRA [32](#) [77](#). This approach drastically reduces the number of trainable parameters, making the process computationally feasible on low-CPU systems while still achieving high performance [77](#). The training process itself is optimized by leveraging all available CPU threads and RAM, and the final model is subjected to quantization (e.g., to INT8 or INT4) to minimize its memory footprint and accelerate inference [5](#) [86](#). The model's performance is validated on a held-out portion of the dataset to ensure it has learned the desired behavior without overfitting.

By following the Curate-and-Focus Methodology, a developer can successfully navigate the complexities of building specialized LLMs under resource constraints. It provides a robust, evidence-based framework that emphasizes quality, discipline, and efficiency. The result is not just a model, but a purpose-built tool, finely attuned to a specific domain and capable of delivering high performance on the very hardware platforms where large-scale models are traditionally infeasible.

Reference

1. An Evaluation of LLMs Inference on Popular Single-board Computers <https://arxiv.org/html/2511.07425v1>
2. Mastering LLM Techniques: Text Data Processing - NVIDIA Developer <https://developer.nvidia.com/blog/mastering-llm-techniques-data-preprocessing/>
3. Characterizing CPU-Induced Slowdowns in Multi-GPU LLM Inference <https://arxiv.org/html/2603.22774v1>
4. (PDF) Investigating Large Language Models' Linguistic Abilities for ... https://www.researchgate.net/publication/396460525_Investigating_Large_Language_Models'_Linguistic_Abilities_for_Text_Preprocessing
5. Running Local LLMs, CPU vs. GPU - a Quick Speed Test <https://dev.to/maximsaplin/running-local-llms-cpu-vs-gpu-a-quick-speed-test-2cjm/comments>
6. Low Computational Cost for Sample Entropy - MDPI <https://www.mdpi.com/1099-4300/20/1/61>
7. Rule-Based Rating and Selection of LLM Training Data - OpenReview <https://openreview.net/forum?id=SpTzsQjgxF>
8. Fine-grained evaluation of a domain-specific Q&A dataset to support ... <https://pmc.ncbi.nlm.nih.gov/articles/PMC13109443/>
9. [PDF] Faithful Persona-based Conversational Dataset Generation with ... <https://aclanthology.org/2024.nlp4convai-1.8.pdf>
10. A Survey on Recent Advances in Conversational Data Generation <https://arxiv.org/html/2405.13003v1>
11. A Survey on Recent Advances in Conversational Data Generation <https://dl.acm.org/doi/full/10.1145/3795686>
12. Data-efficient LLM Fine-tuning for Code Generation - arXiv <https://arxiv.org/html/2504.12687v1>
13. Data $\times$ LLM: From Principles to Practices - arXiv <https://arxiv.org/html/2505.18458v2>
14. 8.2. Text Preprocessing - Dive into Deep Learning https://classic.d2l.ai/chapter_recurrent-neural-networks/text-preprocessing.html

15. Data Preprocessing Techniques for LLMs | PDF | Learning - Scribd <https://www.scribd.com/presentation/960503579/5-Data-Preprocessing-for-LLMs>
16. Fine-Tuning LLMs using Intel Xeon CPUs - Lenovo Press <https://lenovopress.lenovo.com/lp2179-fine-tuning-llms-using-intel-xeon-cpus>
17. Building a small LLM for Raspberry Pi: Fine-Tuning vs RAG - LinkedIn https://www.linkedin.com/posts/bhaveshtgupta_ai-llm-finetuning-activity-7371537970501242880-UppI
18. Creating Conversational Datasets for Retrieval-Augmented ... <https://dl.acm.org/doi/10.1145/3706599.3719962>
19. Conversational AI Design: Best Practices from Zoolatech <https://zoolatech.com/blog/conversational-ai-design-best-practices/>
20. [PDF] Data, Data Everywhere: A Guide for Pretraining Dataset Construction <https://aclanthology.org/2024.emnlp-main.596.pdf>
21. When CPUs Outperform for On-Device LLM Inference - arXiv <https://arxiv.org/html/2505.06461v1>
22. [PDF] Benchmarking the Power Consumption of LLM Inference <https://ojs.aaai.org/index.php/AAAI/article/view/40535/44496>
23. BLISS: A Lightweight Bilevel Influence Scoring Method for Data ... <https://arxiv.org/html/2510.06048v1>
24. 1 Introduction - arXiv <https://arxiv.org/html/2510.06048v4>
25. The Devil is in the Training Data <https://blog.genlaw.org/explainers/training-data.html>
26. Fine-Tuning Large Language Models with Domain-Specific Data https://www.researchgate.net/publication/397139167_Fine-Tuning_Large_Language_Models_with_Domain-Specific_Data_A_Comprehensive_Analysis_of_Parameter-Efficient_Methods_and_Performance_Optimization
27. Fine-tuning of Large Language Models for Domain-Specific ... - arXiv <https://arxiv.org/html/2509.25241v1>
28. An efficient strategy for fine-tuning large language models - Frontiers <https://www.frontiersin.org/journals/artificial-intelligence/articles/10.3389/frai.2026.1665992/full>
29. Domain Specialization as the Key to Make Large Language Models ... <https://dl.acm.org/doi/full/10.1145/3764579>
30. (PDF) Low-Resource Fine-Tuning of LLMs for Domain-Specific Tasks https://www.academia.edu/144631226/Low_Resource_Fine_Tuning_of_LLMs_for_Domain_Specific_Tasks

31. A Survey of Domain-Specific Fine-Tuned Large Language Models <https://ieeexplore.ieee.org/iel8/6287639/11323511/11364219.pdf>
32. Low-Resource Fine-Tuning of LLMs for Domain-Specific Tasks <https://www.semanticscholar.org/paper/Low-Resource-Fine-Tuning-of-LLMs-for-Tasks-Challa-Bright/f19b3348540965c1c2415e4caciaa009483afff15>
33. A Comprehensive Survey of Small Language Models in the Era of ... <https://dl.acm.org/doi/10.1145/3768165>
34. (PDF) Computational Cost Estimation in LLMs: A Comparative Study ... https://www.researchgate.net/publication/400336962_Computational_Cost_Estimation_in_LLMs_A_Comparative_Study_between_Open-Chats_and_Benchmarks
35. A Survey on Recent Advances in Conversational Data Generation <https://dl.acm.org/doi/10.1145/3795686>
36. NaturalConv: A Chinese Dialogue Dataset Towards Multi-turn Topic ... <https://arxiv.org/html/2103.02548v3>
37. [PDF] CONV-TO-BENCH: EVALUATING LANGUAGE MODELS <https://openreview.net/pdf?id=PmkOtKBMr0>
38. [PDF] Deep learning for code generation: a survey <http://scis.scichina.com/en/2024/191101.pdf>
39. [PDF] Building Open-Retrieval Conversational Question Answering ... <https://aclanthology.org/2025.sigdial-1.52.pdf>
40. [PDF] Pretraining Context Compressor for Large Language Models with ... https://csse.szu.edu.cn/attachment/cglr/1763439922_2025.acl-long.1394.pdf
41. Improving AI Domain Code Generation of Large Language Models <https://arxiv.org/html/2406.04712v1>
42. Survey on Code Generation for Low resource and Domain Specific ... https://www.researchgate.net/publication/384700235_Survey_on_Code_Generation_for_Low_resource_and_Domain_Specific_Programming_Languages
43. A Framework for Domain-Specific Dataset Creation and Adaptation ... <https://www.mdpi.com/2073-431X/14/5/172>
44. On the Effectiveness of Large Language Models in Domain-Specific ... <https://dl.acm.org/doi/full/10.1145/3697012>
45. [PDF] Domain Adaptation for Dense Retrieval and Conversational Dense ... <https://aclanthology.org/2024.lrec-main.467.pdf>
46. [PDF] CoderGen: Towards Domain-Specific Code Generation of Large ... <https://openreview.net/pdf?id=7fo7yixup1>

47. View of ChEdBot: Designing a Domain-Specific Conversational ... <https://ojs.aaai.org/index.php/AAAI-SS/article/view/31198/33358>
48. Fine-tuning large language models for domain adaptation - Nature <https://www.nature.com/articles/s41524-025-01564-y>
49. Cost-Performance Trade-Off Analysis of Training vs. Fine- Tuning ... https://www.researchgate.net/publication/398712238_Cost-Performance_Trade-Off_Analysis_of_Training_vs_Fine-_Tuning_LLMs_on_Budget_Hardware
50. [PDF] LiteRT-Optimized INT8 LLM for Raspberry Pi4 Deployment https://openaccess.thecvf.com/content/ICCV2025W/AIM/papers/Yoon_LiteRT-Optimized_INT8_LLM_for_Raspberry_Pi4_Deployment_ICCVW_2025_paper.pdf
51. [PDF] Fine-Tuning LLMs using Intel Xeon CPUs - Lenovo Press <https://lenovopress.lenovo.com/lp2179.pdf>
52. Fine-grained evaluation of a domain-specific Q&A dataset to support ... <https://link.springer.com/article/10.1007/s13755-026-00458-7>
53. LLMs performance bottleneck: memory bandwidth not capacity https://www.linkedin.com/posts/marcosheidemann_memory-transfer-speed-bottleneck-is-real-activity-7379877805645352961-XGQb
54. [PDF] Entropy-Based Data Selection for Language Models - ResearchGate https://www.researchgate.net/publication/395230153_Entropy-Based_Data_Selection_for_Language_Models/fulltext/68b8047f653c451cf473fe17/Entropy-Based-Data-Selection-for-Language-Models.pdf
55. Knowledge distillation and dataset distillation of large language ... <https://link.springer.com/article/10.1007/s10462-025-11423-3>
56. The Largest Collection of Ethical Data for LLM Pre-Training - arXiv <https://arxiv.org/html/2506.01732v3>
57. Scale and Curate High-Quality Datasets for LLM Training with ... <https://developer.nvidia.com/blog/scale-and-curate-high-quality-datasets-for-llm-training-with-nemo-curator/>
58. LLM-Assisted Code Cleaning For Training Accurate Code Generators <https://openreview.net/forum?id=maRYffiUpI>
59. The Ultimate Guide to Fine-Tuning LLMs from Basics to Breakthroughs <https://arxiv.org/html/2408.13296v1>
60. Data-Centric Fine-Tuning of Small Language Models for Automatic ... <https://ieeexplore.ieee.org/iel8/6287639/10820123/11088115.pdf>
61. Fine-tuning large language models (LLMs) in 2026 - SuperAnnotate <https://www.superannotate.com/blog/llm-fine-tuning>
62. Fine-tuning protein language models boosts predictions ... - Nature <https://www.nature.com/articles/s41467-024-51844-2>

63. Parameter-efficient fine-tuning in large language models: a survey of ... <https://link.springer.com/article/10.1007/s10462-025-11236-4>
64. [PDF] Fine-tuning Language Models on Dell PowerEdge XE9680 Servers <https://www.delltechnologies.com/asset/en-us/products/servers/industry-market/finetuning-llm-on-xe9680.pdf>
65. A Comprehensive Overview of Large Language Models <https://dl.acm.org/doi/full/10.1145/3744746>
66. Medical LLMs: Fine-Tuning vs. Retrieval-Augmented Generation <https://pmc.ncbi.nlm.nih.gov/articles/PMC12292519/>
67. Benchmarking Large Language Model Inference on Limited ... - MDPI <https://www.mdpi.com/2079-9292/15/11/2451>
68. Benchmarking AI inference on CPUs <https://next.redhat.com/2026/05/28/benchmarking-ai-inference-on-cpus-a-transparent-blueprint-for-the-enterprise/>
69. Code2Doc: A Quality-First Curated Dataset for Code Documentation <https://arxiv.org/html/2512.18748v1>
70. On the Effectiveness of Large Language Models in Domain-Specific ... <https://dl.acm.org/doi/10.1145/3697012>
71. [PDF] An Evolving Code Generation Benchmark with Domain-Specific ... <https://openreview.net/pdf?id=kvjbfVHpnny>
72. [PDF] Constructing Domain-Specific Evaluation Sets for LLM-as-a-judge <https://aclanthology.org/2024.customnlp4u-1.14.pdf>
73. Large Language Model Performance Benchmarking on Mobile ... <https://arxiv.org/html/2410.03613v1>
74. Entropy, Volume 28, Issue 5 (May 2026) – 99 articles - MDPI <https://www.mdpi.com/1099-4300/28/5>
75. [PDF] Soft Label Compression-Centric Dataset Condensation via Coding ... https://openaccess.thecvf.com/content/WACV2026/papers/Yuan_SCORE_Soft_Label_Compression-Centric_Dataset_Condensation_via_Coding_Rate_Optimization_WACV_2026_paper.pdf
76. How to Create Data for Fine-Tuning LLMs | DigitalOcean <https://www.digitalocean.com/community/tutorials/how-to-create-llm-finetuning-dataset>
77. A Survey on LLM-based Code Generation for Low-Resource ... - arXiv <https://arxiv.org/html/2410.03981v3>
78. Toxicity of the Commons: Curating Open-Source Pre-Training Data <https://arxiv.org/html/2410.22587v2>

79. [PDF] AUDITING TEST DATA CONTAMINATION WITH ERROR RATE ... <https://openreview.net/pdf?id=YsoabhpS7z>
80. Investigating Data Contamination in Modern Benchmarks for Large ... https://www.researchgate.net/publication/382633331_Investigating_Data_Contamination_in_Modern_Benchmarks_for_Large_Language_Models
81. LLMPi: Optimizing LLMs for High-Throughput on Raspberry Pi - arXiv <https://arxiv.org/html/2504.02118v1>
82. Vol. 40 No. 36: AAAI-26 Technical Tracks 36 <https://ojs.aaai.org/index.php/AAAI/issue/view/718>
83. Top LLM Papers of the Week (June Week-4 2024) - LinkedIn <https://www.linkedin.com/pulse/top-llm-papers-week-june-week-4-2024-kalyan-ks-jenyc>
84. [PDF] LLMPi: Optimizing LLMs for High-Throughput on Raspberry Pi https://openaccess.thecvf.com/content/CVPR2025W/EDGE/papers/Ardakani_LLMPi_Optimizing_LLMs_for_High-Throughput_on_Raspberry_Pi_CVPRW_2025_paper.pdf
85. [PDF] Structured Packing in LLM Training Improves Long Context Utilization <https://ojs.aaai.org/index.php/AAAI/article/view/34706/36861>
86. AI performance research papers - Red Hat <https://www.redhat.com/en/artificial-intelligence/research>
87. [PDF] Domain-Specific Embedding Models for Hydrology and ... - EarthArXiv <https://eartharxiv.org/repository/object/9635/download/19581/>
88. Exploring the effectiveness of instruction tuning in biomedical ... - PMC <https://pmc.ncbi.nlm.nih.gov/articles/PMC12040708/>
89. A Benchmark Dataset for Open-domain Dialogue Quality Assessment <https://openreview.net/forum?id=3ErwybEDgt-eId=dng4sJaCqW>
90. [PDF] DialogStudio: Towards Richest and Most Diverse Unified Dataset ... <https://aclanthology.org/2024.findings-eacl.152.pdf>
91. [PDF] Building Domain-Specific Small Language Models via Guided Data ... <https://ojs.aaai.org/index.php/AAAI/article/view/41467/45428>
92. doqa - OpenDataLab <https://opendatalab.com/OpenDataLab/doqa>
93. [2312.10007] Faithful Persona-based Conversational Dataset ... - ar5iv <https://ar5iv.labs.arxiv.org/html/2312.10007>
94. [PDF] Faithful Persona-based Conversational Dataset Generation ... <https://www.semanticscholar.org/paper/Faithful-Persona-based-Conversational-Dataset-with-Jandaghi-Sheng/a86d3692af34ce70fd8caaa319009f85a88fdb4d>

95. Bias in the Loop: Auditing LLM-as-a-Judge for Software Engineering <https://arxiv.org/html/2604.16790v1>
96. Code2Doc: A Quality-First Curated Dataset for Code Documentation https://www.researchgate.net/publication/398980230_Code2Doc_A_Quality-First_Curated_Dataset_for_Code_Documentation
97. Running Local LLMs, CPU vs. GPU - a Quick Speed Test <https://dev.to/maximsaplin/running-local-llms-cpu-vs-gpu-a-quick-speed-test-2cjm>
98. OKBench: Democratizing LLM Evaluation with Fully Automated, On ... <https://arxiv.org/html/2511.08598v1>
99. [PDF] A Thorough Investigation of Content-Defined Chunking Algorithms ... <https://arxiv.org/pdf/2409.06066>
100. Automatic Dataset Generation for Knowledge Intensive Question ... <https://arxiv.org/html/2505.14212v1>
101. [PDF] LiveChat: A Large-Scale Personalized Dialogue Dataset ... <https://aclanthology.org/2023.acl-long.858.pdf>
102. Dialogue Agents 101: A Beginner's Guide to Critical Ingredients for ... <https://arxiv.org/html/2307.07255v2>
103. KAT-Coder Technical Report - arXiv <https://arxiv.org/html/2510.18779v1>
104. [PDF] Dialogue Chain-of-Thought Distillation for Commonsense-aware ... <https://aclanthology.org/2023.emnlp-main.342.pdf>
105. ISO/IEC 5259-2:2024(en), Artificial intelligence — Data quality for ... <https://www.iso.org/obp/ui/en/#iso:std:81860:en>
106. LLM-Assisted Code Cleaning For Training Accurate Code Generators <https://www.semanticscholar.org/paper/e1b7f13051bec4a713c6226c177b63599d49e3e9>
107. What Do Developers Think About Code Readability in the Age of ... <https://www.atlassian.com/blog/development/atlassian-research-developers-on-code-readability-llm>
108. Auditing the Shadows: A Review of Methods to Detect Shared ... <https://dl.acm.org/doi/10.1145/3774897>
109. Verifiable by Design: Aligning Language Models to Quote from Pre ... <https://aclanthology.org/2025.naacl-long.191/>
110. Iso-Iec 5259-2,2024 | PDF - Scribd <https://www.scribd.com/document/868732734/ISO-IEC-5259-2-2024>
111. Quality Assurance of LLM-generated Code: Addressing Non ... - arXiv <https://arxiv.org/html/2511.10271v1>
112. Faithful Persona-based Conversational Dataset Generation ... - arXiv <https://arxiv.org/html/2312.10007v1>

113. [PDF] Learning to Memorize Entailment and Discourse Relations for ... <https://ojs.aaai.org/index.php/AAAI/article/view/26489/26261>
114. Summary of our human annotation guidelines - ResearchGate https://www.researchgate.net/figure/Summary-of-our-human-annotation-guidelines_fig1_361060480
115. Human Annotated Dialogues Dataset for Natural Conversational ... <https://www.mdpi.com/2076-3417/10/3/762>
116. Data × LLM: From Principles to Practices - arXiv <https://arxiv.org/html/2505.18458v3>
117. Generating, retrieving persona and generating responses for long ... <https://pmc.ncbi.nlm.nih.gov/articles/PMC12453736/>
118. CODEGEN: A Transformative Open-Source Language Model for ... <https://www.digitalocean.com/community/tutorials/codegen-open-source-language-model-for-code-generation>
119. Revisiting Contextual Toxicity Detection in Conversations <https://dl.acm.org/doi/10.1145/3561390>
120. Open-Domain Dialogue Generation: What We Can Do, Cannot Do ... https://www.researchgate.net/publication/361069030_Open-Domain_Dialogue_Generation_What_We_Can_Do_Cannot_Do_and_Should_Do_Next
121. Empirical Guidelines for Deploying LLMs onto Resource ... <https://dl.acm.org/doi/10.1145/3736721>
122. Distributed training of large language models: A survey <https://www.sciencedirect.com/science/article/pii/S2949719125000500>
123. Densing law of LLMs | Nature Machine Intelligence <https://www.nature.com/articles/s42256-025-01137-0>
124. LLM Papers Reading Notes - May 2025 - LinkedIn <https://www.linkedin.com/pulse/llm-papers-reading-notes-may-2025-jean-david-ruvini-w2gjc>
125. ISO/IEC/IEEE 12207:2026(en), Systems and software engineering <https://www.iso.org/obp/ui/en/#iso:std:90219:en>
126. IEEE/ISO/IEC 12207-2026 - Accuris Standards Store https://store.accuristech.com/standards/ieee-iso-iec-12207-2026?product_id=2918214&srsltid=AfmBOooGglGrstEoGKo179Fz3u3RVqP45wFo-vuAsh_cY4Q9RSIK6BZO
127. Setting up a local LLM on Raspberry Pi 5 with Ollama - LinkedIn https://www.linkedin.com/posts/mrjohnkendrick_air-gapped-llm-blog-2-part-1-setting-up-activity-7448056811888422912-VYu1
128. Analysis of the performance of LLM on the Raspberry Pi 5 - RPubS <https://rpubs.com/harpomaxx/rpi5llm>

129. Challenge Me: Enhancing Conversational Consistency of LLMs by... <https://openreview.net/forum?id=SMKgohbroH>
130. (PDF) Analyzing LLM-generated code according to four ISO/IEC ... https://www.researchgate.net/publication/397956156_Analyzing_LLM-Generated_Code_According_To_Four_ISOIEC_50552021_Categories
131. IEEE/ISO/IEC 12207-2026 - Accuris Standards Store https://store.accuristech.com/standards/ieee-iso-iec-12207-2026?product_id=2918214&srsltid=AfmBOoodQ-inF9RPQbKWBFpk39G34gnD-haQyrbBah7MBrheECCMUAm
132. Data Curation — NVIDIA NeMo Framework User Guide <https://docs.nvidia.com/nemo-framework/user-guide/24.07/datacuration/index.html>
133. Nemo Curator — GPU-accelerated data curation for LLM training <https://hermes-agent.nousresearch.com/docs/user-guide/skills/optional/mlops/mlops-nemo-curator>
134. nemo-curator - PyPI <https://pypi.org/project/nemo-curator/>
135. ISO/IEC/IEEE CD 29148 - Systems and software engineering <https://www.iso.org/standard/94091.html>
136. IEEE 29148-软件工程中的需求工程流程- 软件和系统工程标准系列 <https://srs.pub/specification/req-flow-guide.html>
137. IEEE/ISO/IEC 12207-2026 - Accuris Standards Store https://store.accuristech.com/standards/ieee-iso-iec-12207-2026?product_id=2918214&srsltid=AfmBOopEwnotZaorujxxSV7JnnEhgYhSaXbNtCYrtK1Gag8aCn0pxxV6
138. A Comparative Analysis using ISO/IEC/IEEE 29148 - ResearchGate https://www.researchgate.net/publication/366639331_User-Centered_Methodologies_for_Requirements_Engineering_A_Comparative_Analysis_using_ISOIECIEEE_29148
139. LLM-REDIAL: A Large-Scale Dataset for Conversational ... <https://aclanthology.org/2024.findings-acl.529/>
140. [PDF] LLM-REDIAL: A Large-Scale Dataset for Conversational ... <https://openreview.net/pdf/26a493e79b063f91c2a32a3985b2f4fdace40ec9.pdf>
141. LLM-REDIAL: A Large-Scale Dataset for Conversational ... https://www.researchgate.net/publication/384215802_LLM-REDIAL_A_Large-Scale_Dataset_for_Conversational_Recommender_Systems_Created_from_User_Behaviors_with_LLMs
142. SlimPajama-627B - ModelScope <https://modelscope.cn/datasets/swift/SlimPajama-627B>
143. SlimPajama-DC: Understanding Data Combinations for LLM Training <https://arxiv.org/html/2309.10818v3>

144. SlimPajama-627B - OpenDataLab <https://opendatalab.com/OpenDataLab/SlimPajama-627B>
145. RedPajama: an Open Dataset for Training Large Language Models [https://openreview.net/forum?id=lnuXaRpww&referrer=%5Bthe%20profile%20of%20Tri%20Dao%5D\(%2Fprofile%3Fid%3D~Tri_Dao1\)](https://openreview.net/forum?id=lnuXaRpww&referrer=%5Bthe%20profile%20of%20Tri%20Dao%5D(%2Fprofile%3Fid%3D~Tri_Dao1))
146. MemoryDocDataSet: A Benchmark for Joint Conversational Memory ... <https://arxiv.org/html/2606.04442v1>
147. Tokenization efficiency of current foundational large language ... <https://www.frontiersin.org/journals/artificial-intelligence/articles/10.3389/frai.2025.1538165/full>
148. [PDF] EXECUTE: A Multilingual Benchmark for LLM Token Understanding <https://aclanthology.org/2025.findings-acl.95.pdf>
149. QualBench: Benchmarking Chinese LLMs with Localized ... - arXiv <https://arxiv.org/html/2505.05225v1>
150. [PDF] AlignBench: Benchmarking Chinese Alignment of Large Language ... <https://aclanthology.org/2024.acl-long.624.pdf>
151. Low-resource Preference Alignment by Weak-to-Strong Decoding <https://arxiv.org/html/2506.07434v1>
152. SlimPajama-DC: Data Combinations for LLM Training <https://hunterheidenreich.com/notes/natural-language-processing/language-models/slimpajama-dc-data-combinations/>
153. SlimPajama-DC: Understanding Data Combinations for LLM Training <https://arxiv.org/abs/2309.10818>
154. CerebrasAI开源最新可商用的高质量大语言模型训练数据集，含6270 ... <https://www.datalearner.com/blog/cerebras-slimpajama-trillion-token-dataset>
155. Deduplicating Training Data Makes Language Models Better <https://aclanthology.org/2022.acl-long.577/>
156. Hugging Face 数据集镜像/SlimPajama-6B - Gitee <https://gitee.com/hf-datasets/SlimPajama-6B>
157. Data × LLM: From Principles to Practices - arXiv <https://arxiv.org/html/2505.18458v1>
158. SlimPajama-DC: Understanding Data Combinations for LLM Training <https://ar5iv.labs.arxiv.org/html/2309.10818>